

TUGAS JURNAL MODUL 13



Disusun Oleh :

Rengganis Tantri Pramudita (2311104065)

Kelas : SE0702

Dosen :

Yudha Islami Sulistya

PROGRAM STUDI SOFTWARE ENGINEERING

DIREKTORAT KAMPUS PURWOKERTO

TELKOM UNIVERSITY PURWOKERTO

2025

1. MENJELASKAN DESIGN PATTERN SINGLETON

A. Berikan salah dua contoh kondisi dimana design pattern “Singleton” dapat digunakan.

Jawab:

- Koneksi Database

Dalam sebuah aplikasi, biasanya hanya dibutuhkan satu koneksi database yang digunakan bersama oleh berbagai bagian program. Singleton digunakan agar hanya satu instance koneksi yang dibuat, sehingga efisien dan konsisten.

- Logging Sistem

Saat sebuah aplikasi mencatat log aktivitas, Singleton memastikan bahwa semua komponen menggunakan instance logger yang sama agar penulisan log lebih terstruktur dan tidak ganda.

B. Berikan penjelasan singkat mengenai langkah-langkah dalam mengimplementasikan design pattern “Singleton”.

Jawab:

- Buat constructor bersifat private, agar tidak bisa dibuat instance dari luar kelas.
- Buat field static untuk menyimpan satu-satunya instance kelas.
- Buat method static (biasanya GetInstance() atau Instance) yang mengembalikan instance dan membuatnya jika belum ada.

C. Berikan tiga kelebihan dan kekurangan dari design pattern “Singleton”.

Kelebihan	Kekurangan
Menghemat sumber daya, karena hanya ada satu instance.	Sulit untuk diuji secara unit test, karena instance tidak bisa diganti (tidak fleksibel).
Kontrol akses terpusat, cocok untuk sistem konfigurasi atau logging.	Melanggar prinsip Single Responsibility, karena kelas juga bertanggung jawab untuk mengatur instance-nya sendiri.
Mudah diimplementasikan, terutama pada aplikasi kecil hingga menengah.	Bisa menyebabkan masalah pada aplikasi multi-threaded jika tidak diatur dengan aman (misalnya race condition).

D. IMPLEMENTASI DAN PEMAHAMAN DESIGN PATTERN OBSERVER

Jawab:

a. PusatDataSingleton.cs

```
using System;
using System.Collections.Generic;
namespace modul13_2311104065
{
    public class PusatDataSingleton
    {
        // Tambahkan tanda tanya (?) agar field ini nullable
        private static PusatDataSingleton? _instance;

        public List<string> DataTersimpan { get; set; }

        // Konstruktor privat
        private PusatDataSingleton()
        {
            DataTersimpan = new List<string>();
        }

        // Method Singleton
        public static PusatDataSingleton GetDataSingleton()
        {
            if (_instance == null)
            {
                _instance = new PusatDataSingleton();
            }
            return _instance;
        }

        // Mengembalikan seluruh data
```

```
public List<string> GetSemuaData()
{
    return DataTersimpan;
}

// Menampilkan seluruh data
public void PrintSemuaData()
{
    Console.WriteLine("Isi Data:");
    foreach (var data in DataTersimpan)
    {
        Console.WriteLine("- " + data);
    }
}

// Menambahkan data
public void AddSebuahData(string input)
{
    DataTersimpan.Add(input);
}

// Menghapus data
public void HapusSebuahData(int index)
{
    if (index >= 0 && index < DataTersimpan.Count)
    {
        DataTersimpan.RemoveAt(index);
    }
    else
    {
        Console.WriteLine("Index tidak valid.");
    }
}
```

```
    }  
    }  
    }  
}
```

b. Program.cs

```
using System;  
namespace modul13_2311104065  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            var data1 = PusatDataSingleton.GetDataSingleton();  
            var data2 = PusatDataSingleton.GetDataSingleton();  
  
            data1.AddSebuahData("Rengganis");  
            data1.AddSebuahData("Suga");  
            data1.AddSebuahData("Jimin");  
            data1.AddSebuahData("Jungkook");  
  
            Console.WriteLine("Data2 sebelum penghapusan:");  
            data2.PrintSemuaData();  
  
            data2.HapusSebuahData(3);  
  
            Console.WriteLine("\nData1 setelah penghapusan dari data2:");  
            data1.PrintSemuaData();  
        }  
    }  
}
```

```
        Console.WriteLine($"Jumlah data (data1):  
        {data1.GetSemuaData().Count}");  
  
        Console.WriteLine($"Jumlah data (data2):  
        {data2.GetSemuaData().Count}");  
  
        Console.WriteLine("\nTekan tombol apa saja untuk keluar...");  
        Console.ReadKey();  
    }  
}  
}
```

Penjelasan:

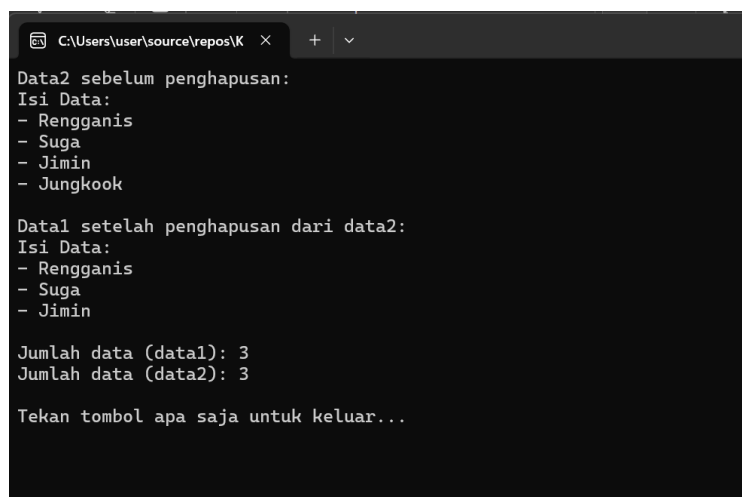
Program ini menggunakan design pattern Singleton melalui class `PusatDataSingleton`, yang menjamin hanya ada satu instance (objek) yang digunakan selama program berjalan.

Di dalam Main, dua variabel (`data1` dan `data2`) mengambil instance yang sama dari `PusatDataSingleton` menggunakan method `GetDataSingleton()`.

Lalu, beberapa data ditambahkan melalui `data1`. Kemudian `data2` digunakan untuk menghapus satu item dari daftar. Karena `data1` dan `data2` mengakses instance yang sama, perubahan dari `data2` juga terlihat saat mencetak isi dari `data1`.

Akhirnya, program mencetak jumlah data dari `data1` dan `data2`, yang hasilnya sama karena keduanya menunjuk ke instance yang sama.

Outputnya



```
C:\Users\user\source\repos\K x + v  
Data2 sebelum penghapusan:  
Isi Data:  
- Rengganis  
- Suga  
- Jimin  
- Jungkook  
  
Data1 setelah penghapusan dari data2:  
Isi Data:  
- Rengganis  
- Suga  
- Jimin  
  
Jumlah data (data1): 3  
Jumlah data (data2): 3  
  
Tekan tombol apa saja untuk keluar...
```